



main.py

 Share  Run

```
1- def linear_search(arr, key):
2-     for i in range(len(arr)):
3-         if arr[i] == key:
4-             return i
5-     return -1
6 arr = [10, 25, 30, 45, 50]
7 key = 30
8 result = linear_search(arr, key)
9- if result != -1:
10     print("Key found at index", result)
11- else:
12     print("Key not found")
```

Output

Clear

Key found at index 2

--- Code Execution Successful ---



main.py



Run

```
1 arr = [-2, -3, 4, -1, -2, 1, 5, -3]
2 max_sum = current = arr[0]
3 for num in arr[1:]:
4     current = max(num, current + num)
5     max_sum = max(max_sum, current)
6 print("Maximum Subarray Sum =", max_sum)
```

Output

Clear

Maximum Subarray Sum = 7

=== Code Execution Successful ===



main.py



Share

Run

Output

Clear

```
1 def merge_count(arr):
2     if len(arr) <= 1:
3         return arr, 0
4     mid = len(arr) // 2
5     left, inv_left = merge_count(arr[:mid])
6     right, inv_right = merge_count(arr[mid:])
7     merged = []
8     i = j = 0
9     inv = inv_left + inv_right
10    while i < len(left) and j < len(right):
11        if left[i] <= right[j]:
12            merged.append(left[i])
13            i += 1
14        else:
15            merged.append(right[j])
16            inv += len(left) - i
17            j += 1
18    merged.extend(left[i:])
19    merged.extend(right[j:])
20    return merged, inv
21 arr = [2, 4, 1, 3, 5]
22 _, count = merge_count(arr)
23 print("Inversions =", count)
```

Inversions = 3

--- Code Execution Successful ---

Programiz

Python Online Compiler

Python Online Compiler

main.py

1 import heapq

2 arr = [4, 10, 3, 5, 1]

3 heapq.heapify(arr)

4 sorted_arr = []

5 while arr:

6 sorted_arr.append(heapq.heappop(arr))

7 print(sorted_arr)

Share

Run

Output

Clear

[1, 3, 4, 5, 10]

--- Code Execution Successful ---



JS

TS

Go



main.py



Share

Run

```
1 arr = [12, 11, 13, 5, 6]
2 for i in range(1, len(arr)):
3     key = arr[i]
4     j = i - 1
5     while j >= 0 and key < arr[j]:
6         arr[j + 1] = arr[j]
7         j -= 1
8     arr[j + 1] = key
9 print(arr)
```

Output

Clear

[5, 6, 11, 12, 13]

=== Code Execution Successful ===



main.py



Share

Run

```
1 def gcd(a, b):
2     if b == 0:
3         return a
4     return gcd(b, a % b)
5 a = 48
6 b = 18
7 print("GCD =", gcd(a, b))
```

Output

Clear

GCD = 6

=== Code Execution Successful ===

main.py

Share

Run

```
1- def tower(n, source, helper, destination):
2-     if n == 1:
3-         print("Move disk 1 from", source, "to", destination)
4-         return
5-     tower(n - 1, source, destination, helper)
6-     print("Move disk", n, "from", source, "to", destination)
7-     tower(n - 1, helper, source, destination)
8- tower(3, 'A', 'B', 'C')
```

Output

Clear

Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C

--- Code Execution Successful ---

JS

TS



main.py

 Share

Run

```
1- def quick_sort(arr):
2-     if len(arr) <= 1:
3-         return arr
4-     pivot = arr[len(arr) // 2]
5-     left = [x for x in arr if x < pivot]
6-     middle = [x for x in arr if x == pivot]
7-     right = [x for x in arr if x > pivot]
8-     return quick_sort(left) + middle + quick_sort(right)
9 arr = [10, 7, 8, 9, 1, 5]
10 print(quick_sort(arr))
```

Output

Clear

[1, 5, 7, 8, 9, 10]

=== Code Execution Successful ===



main.py



Share

Run

Output

Clear

```
1- def merge_sort(arr):
2-     if len(arr) > 1:
3-         mid = len(arr) // 2
4-         left = arr[:mid]
5-         right = arr[mid:]
6-         merge_sort(left)
7-         merge_sort(right)
8-         i = j = k = 0
9-         while i < len(left) and j < len(right):
10-             if left[i] < right[j]:
11-                 arr[k] = left[i]
12-                 i += 1
13-             else:
14-                 arr[k] = right[j]
15-                 j += 1
16-             k += 1
17-         while i < len(left):
18-             arr[k] = left[i]
19-             i += 1
20-             k += 1
21-         while j < len(right):
22-             arr[k] = right[j]
23-             j += 1
24-             k += 1
25 arr = [38, 27, 43, 3, 9, 82, 10]
26 merge_sort(arr)
27 print(arr)
```

[3, 9, 10, 27, 38, 43, 82]

=== Code Execution Successful ===



JS

TS

main.py



Share

Run

```
1 A = [[1, 2],
2      [3, 4]]
3 B = [[5, 6],
4      [7, 8]]
5 result = [[0, 0], [0, 0]]
6 for i in range(len(A)):
7     for j in range(len(B[0])):
8         for k in range(len(B)):
9             result[i][j] += A[i][k] * B[k][j]
10 print("Result:")
11 for row in result:
12     print(row)
```

Output

Clear

Result:
[19, 22]
[43, 50]

=== Code Execution Successful ===



main.py



Run

```
1~ def factorial_iterative(n):
2~     fact = 1
3~     for i in range(1, n + 1):
4~         fact *= i
5~     return fact
6
7~ def factorial_recursive(n):
8~     if n == 0 or n == 1:
9~         return 1
10~    return n * factorial_recursive(n - 1)
11~ n = 5
12~ print("Iterative:", factorial_iterative(n))
13~ print("Recursive:", factorial_recursive(n))
```

Output

Clear

Iterative: 120
Recursive: 120

=== Code Execution Successful ===